Name bhavya rana
Sapid 590034689
Batch 15

Algorithm

1. Read the string `s`.
2. Pass its address to `longest_unique_substring()`.
3. Find the length of the input string.
4. If the string is empty, display `0`.
5. For each character, consider it as the starting position of a substring.
6. Move toward the right one character at a time.
7. For each new character, check whether it already appears in the current substring.
8. If a repeated character is found, stop extending that substring.
9. Keep track of the maximum substring length found.
10. Display the maximum length.

Test Case Table

Test	Input	Longest Unique Substring Example	Expected Output
1	`abcabcbb`	`abc`	`Length of longest substring: 3`
2	`bbbbb`	`b`	`Length of longest substring: 1`
3	`pwwkew`	`wke`	`Length of longest substring: 3`
4	Empty string	—	0
5	`abcdef`	`abcdef`	`Length of longest substring: 6`
6	`aA`	`aA`	`Length of longest substring: 2`
7	`abba`	`ab` or `ba`	`Length of longest substring: 2`

C program

```
#include <stdio.h>
#include <string.h>

void longest_unique_substring(char *s)
{
    int i, j, k;
    int length = strlen(s);
    int maxLength = 0;
    int currentLength;

    /* Empty string */
    if (length == 0)
    {
        printf("0\n");
        return;
    }

    /* Check every possible starting position */
    for (i = 0; i < length; i++)
    {
        currentLength = 0;

        for (j = i; j < length; j++)
        {
            int repeated = 0;

            /* Check whether s[j] already exists
               in the current substring */
            for (k = i; k < j; k++)
            {
                if (*(s + k) == *(s + j))
                {
                    repeated = 1;
                    break;
                }
            }

            if (repeated)
                break;

            currentLength++;

            if (currentLength > maxLength)
                maxLength = currentLength;
        }
    }

    printf("Length of longest substring: %d\n", maxLength);
}

int main()
{
    char s[1000];

    printf("Enter a string: ");
    fgets(s, sizeof(s), stdin);
```

```
    /* Remove newline added by fgets() */
    s[strcspn(s, "\n")] = '\0';

    longest_unique_substring(s);

    return 0;
}
```

Execution

